Introduction

The Transformer architecture (Vaswani et al., 2017) replaced recurrent networks as the dominant sequence model. Its core mechanism is **self-attention**: each token attends to every other token in the sequence with learned weights. Transformers underpin BERT, GPT, T5, and modern LLMs, and have expanded into vision (ViT), audio (wav2vec2), and tabular domains.

Prerequisites

Neural networks; attention concept; tokenisation basics.

Theory

Scaled dot-product attention:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V,$$

where Q, K, V are query, key, and value projections of the input.

Multi-head attention runs h parallel attention heads and concatenates outputs.

Transformer block = multi-head self-attention + feedforward + residual + layer norm, stacked many times.

Positional encoding injects sequence order (sinusoidal or learned).

Assumptions

Sufficient data (transformers are data-hungry); proper tokenisation (BPE or WordPiece for text); positional encoding matches task.

R Implementation

```
library(torch)

torch_manual_seed(2026)

# Minimal transformer block: multi-head attention + feedforward
block <- nn_module(
  initialize = function(d_model = 32, n_heads = 4, d_ff = 64) {
    self$mha <- nn_multihead_attention(embed_dim = d_model,
                                       num_heads = n_heads,
                                       batch_first = TRUE)
    self$ln1 <- nn_layer_norm(d_model)
    self$ff  <- nn_sequential(nn_linear(d_model, d_ff),
                              nn_relu(),
                              nn_linear(d_ff, d_model))
    self$ln2 <- nn_layer_norm(d_model)
  },
  forward = function(x) {
    attn <- self$mha(x, x, x)[[1]]
    x <- self$ln1(x + attn)
    x <- self$ln2(x + self$ff(x))
    x
  }
)
```

```
model <- block()
seq <- torch_randn(2, 10, 32)        # batch=2, seq=10, d_model=32
out <- model(seq)
out$shape                             # [2, 10, 32]
```

Output & Results

Output sequence of the same shape as the input; the block transforms representations by letting each token attend to the full context.

Interpretation

"A single transformer block with 4-head self-attention and feedforward sublayers transformed a (10-token, 32-dim) sequence. Stacking 6-48 such blocks gives modern NLP models; pretrained backbones (BERT, RoBERTa) are the practical starting point."

Practical Tips

- Transformers need a lot of data; for small-data tasks, fine-tune a pretrained backbone rather than training from scratch.
- Use efficient attention variants (FlashAttention, sparse attention) for long sequences.
- Masking (causal for GPT, padding for BERT) is essential; forgetting it silently breaks training.
- Positional encoding: sinusoidal for classical transformers, rotary (RoPE) for modern LLMs.
- For R, practical transformer pipelines often call HuggingFace via `reticulate`; `torch` in R implements building blocks but lacks a huge pretrained-model ecosystem.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis